

DEPARTMENT OF COMPUTER SCIENCE FALL 2025

**CS4110 - High Performance
Computing with GPUs**

**Deliverable 4
Report**

By:

Fatima Sohail [23i-0633]

Hareem Zia [23f-0842]

HPC-A

Table Of Contents

Abstract	2
1. Introduction	2
2. Methodology	2
3. Results	3
Fig 3.1 - Average Execution Times for Each Version on Each Dataset	3
Fig 3.2 - Average Speedup per Dataset	4
Fig 3.3 - Device and Host Communication Times	4
Fig 3.4 - Output Images With Features Tracked	4
4. Discussion	5
5. Conclusion	6

Abstract

This study explores the staged acceleration of the Kanade-Lucas-Tomasi (KLT) feature tracking algorithm on GPUs. Starting from a CPU baseline, we identify computational hotspots, then progressively implement GPU acceleration using naive CUDA, optimized CUDA, and OpenACC directives. Performance analysis across multiple datasets and resolutions shows speedups ranging from 2.13x to 24.95x, with the highest gains achieved by fully optimized CUDA kernels using shared memory, persistent device memory, and pipeline restructuring. OpenACC provides simpler GPU offloading with moderate speedup (1.7-3x). The study highlights the trade-offs between implementation complexity and performance, providing a framework for efficient GPU acceleration of feature-tracking pipelines.

1. Introduction

This project focuses on accelerating the Kanade–Lucas–Tomasi (KLT) feature-tracking pipeline by progressively optimizing its computational bottlenecks using GPU-based parallelism. The KLT pipeline involves operations such as image convolution, gradient computation, pyramid construction, and feature tracking, all of which are highly parallelizable but computationally expensive on the CPU. Because our input consists of large grayscale images and multi-level image pyramids, the execution cost grows significantly with higher resolutions and larger numbers of tracked features.

The objective of this project is to evaluate and improve the performance of these stages using modern GPU acceleration techniques. We measure correctness, speedup, kernel execution time, and occupancy across all versions. Finally, we compare the optimized CUDA version with directive-based OpenACC implementations to assess programmability, maintainability, and performance trade-offs.

Overall, the project demonstrates a complete workflow: identifying bottlenecks in a real computer-vision algorithm, offloading them to the GPU, applying optimizations, and analysing the resulting performance and GPU utilisation.

2. Methodology

Our methodology followed an optimization approach designed to evaluate how progressively greater GPU control affects the performance of the KLT feature tracking pipeline. The key steps are summarized below:

1. Baseline Profiling (V1)
 - a. Executed the original CPU implementation on datasets ranging from 240p to 1080p.
 - b. Identified `convolveImageHoriz` and `convolveImageVert` as the dominant bottlenecks (~80% of runtime).
 - c. Observed millions of calls to interpolation and gradient functions within `trackFeatures()`, highlighting additional parallelism potential.
2. Naive GPU Port (V2)
 - a. Directly ported the convolution and tracking functions to CUDA kernels.

- b. Used simple grid/block configurations and default memory access patterns.
 - c. Aimed to measure raw benefits of parallelism without architectural optimizations.
- 3. Optimized GPU Implementation (V3)
 - a. Applied architecture-aware techniques including:
 - i. Shared-memory tiling for horizontal and vertical convolution
 - ii. Constant memory for filter coefficients
 - iii. Persistent GPU-resident image pyramids to reduce transfers
 - iv. Batching feature-processing loops to reduce kernel-launch overhead
 - v. Tuned block sizes for improved occupancy and coalesced memory access.
- 4. Directive-Based Acceleration (V4)
 - a. Employed OpenAcc directives in porting the convolve functions (convolveHoriz and convolveVert) to the GPU.
 - b. Relied on compiler-managed memory movement and kernel generation.
 - c. Evaluated as a high-level, minimal-effort alternative to CUDA.

All four versions were tested on identical datasets, and performance was compared using total execution time (without ppmWrites), kernel timings, GPU occupancy, and host-device transfer measurements to assess efficiency and scalability across implementations.

3. Results

Below are the results after the implementation of all versions.

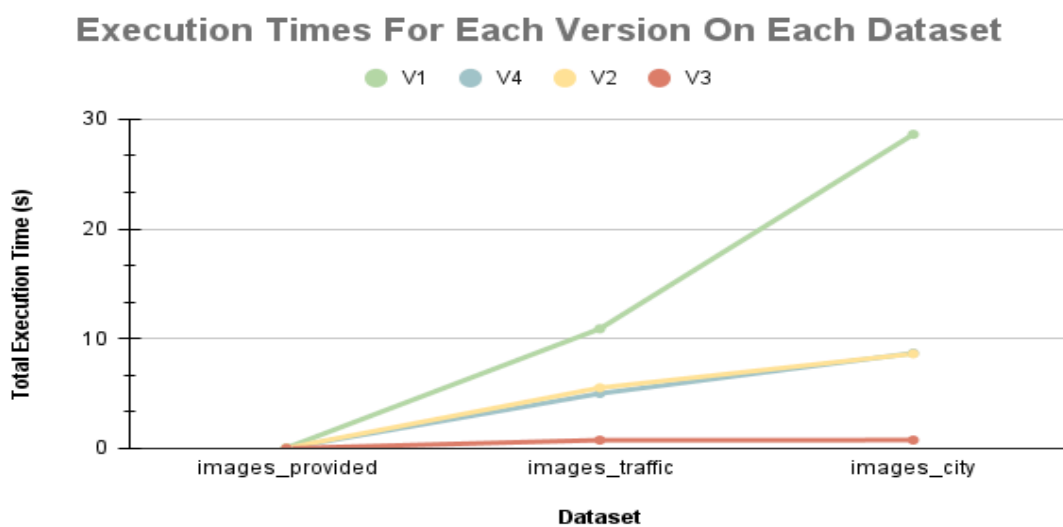


Fig 3.1 - Average Execution Times for Each Version on Each Dataset

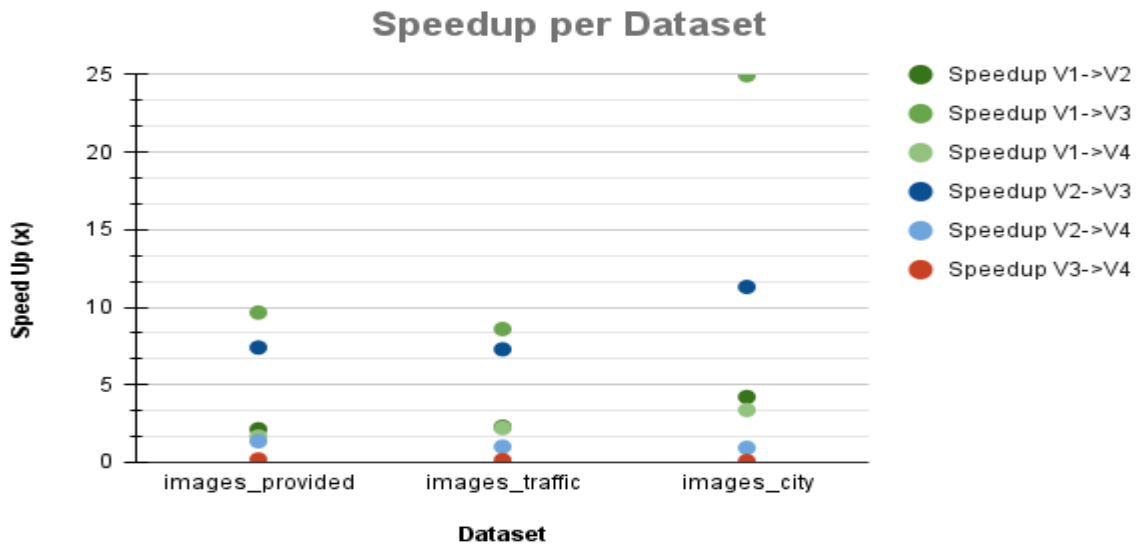


Fig 3.2 - Average Speedup per Dataset

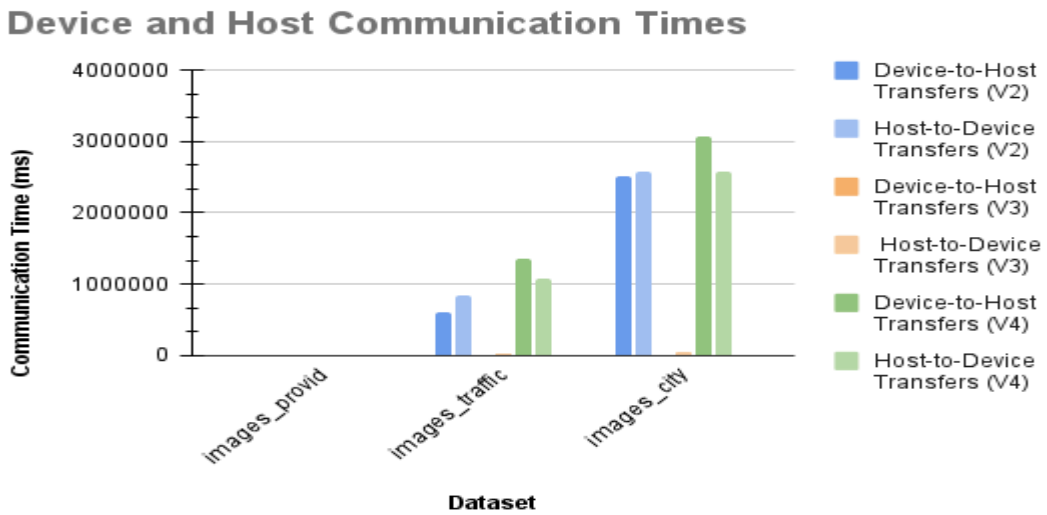


Fig 3.3 - Device and Host Communication Times



Fig 3.4 - Output Images With Features Tracked

The experimental results, detailed across the provided graphs, establish a clear and consistent performance hierarchy: V3 (Optimized CUDA) is the fastest, followed by V2 (Naive CUDA), then V4 (OpenACC), with V1 (CPU) being the slowest by a significant margin. This ranking is immediately evident in *fig.3.1*, which shows the total execution time for each implementation. V1's bar is the tallest for every dataset, while V3's bar is consistently the shortest, confirming its status as the peak performer. The *fig.3.2* graph quantifies this performance delta. It shows that the speedup for V1→V3 (the solid blue bar) is unambiguously the highest for all three datasets, reaching a maximum of ~25x on the 'images_city' dataset.

The *fig.3.2* provides further granularity, illustrating the incremental improvements between versions. It reveals that the single largest performance jump for the 'images_city' dataset occurred between V2 and V3, underscoring the massive impact of the CUDA optimizations. The performance gap between V3 and V4 is also visible here. *Fig.3.3* suggests that data transfer overhead is a critical factor. The superior performance of V3 implies that its manual optimizations successfully minimized this latency, an advantage that the more automated OpenACC approach of V4 could not fully replicate, resulting in its second-place finish.

4. Discussion

The results across Versions 1- 4 highlight how GPU parallelization techniques, memory management, and kernel design impact KLT feature tracking performance. Moving from CPU to naive CUDA, optimized CUDA, and OpenACC showed us, both the potential and limits of GPU acceleration for convolution and feature-intensive workloads.

CPU Baseline (V1): Profiling confirmed that over 80% of runtime spent in `convolveImageHoriz` and `convolveImageVert`. Feature tracking also generated millions of interpolation and gradient calls, indicating high suitability for GPU acceleration if memory and data transfers are efficiently handled.

Naive CUDA (V2): Offloading computation achieved moderate speedups (2.13x - 4.21x) but was limited by excessive CPU–GPU data transfers, high kernel-launch frequency, lack of shared memory, and low occupancy in `trackFeatures`.

Optimized CUDA (V3): Targeted optimizations, e.g. shared-memory tiling, constant memory for filters, persistent GPU-resident pyramids, batched feature kernels, and warp-friendly block/grid sizes, produced substantial speedups (9.65x–24.95x). These changes shifted the bottleneck from communication to computation, fully leveraging GPU resources and illustrating that high-performance tracking requires both kernel and pipeline redesign.

OpenACC (V4): OpenACC-based optimization simplified development and improved portability, achieving moderate speedups (1.73x - 3.01x). However, limited memory control, high data transfers between CPU and GPU, and compiler-managed scheduling prevented performance from matching V3. OpenACC is effective for rapid parallelization but cannot replace architecture-specific optimization for demanding KLT workloads.

Trade-offs between V3 and V4: The comparison between the optimized CUDA implementation (V3) and the OpenACC-based approach (V4) highlights a clear trade-off between peak performance and programming simplicity. V3 achieves significantly higher speedups, reaching 9.65x to 24.95x across our datasets, due to explicit control over shared memory, constant memory usage, persistent GPU-resident data, and carefully tuned block and grid configurations. These optimizations enable high occupancy, reduced memory latency, and minimal CPU–GPU communication, especially in the convolution and feature-tracking stages. However, this level of performance comes at the cost of substantial implementation effort, with V3 requiring manual kernel design, careful memory management, restructuring of the computation pipeline, and detailed profiling to exploit GPU architectural features effectively.

In contrast, V4 provides a much simpler and more portable parallelization path by relying on OpenACC directives that automatically offload computation to the GPU with minimal code changes. While this greatly improves development productivity and maintainability, the lack of fine-grained memory and scheduling control limits performance; V4 achieves only 1.7x to 3.0x speedups and suffers from high device–host transfer overheads and suboptimal memory reuse. As a result, V4 is better suited for rapid prototyping or cross-platform portability, whereas V3 is essential for applications requiring high-throughput, real-time KLT tracking.

Comparative Insights: Larger datasets achieve greater speedups due to amortized overheads, memory management dominates performance, directive-based approaches improve portability but not peak performance, and the KLT algorithm remains highly parallel yet memory-bound, with convolution and gradient computations dominating runtime even on GPUs.

5. Conclusion

This research illustrates a systematic approach to accelerating the KLT feature tracking algorithm on GPUs. Through profiling, naive CUDA, optimized CUDA, and OpenACC implementations, we achieved speedups ranging from 0.52x to 24.95x. Optimized CUDA provides maximal performance, while OpenACC allows rapid GPU offloading with moderate speedup. The staged methodology demonstrates the balance between code complexity, maintainability, and computational efficiency, offering a reference framework for future GPU-accelerated computer vision applications.